Práctica: Desarrollo Guiado por Pruebas (TDD)

Sección 1: Explicación de TDD

Desarrollo Guiado por Pruebas (TDD - Test Driven Development) es una metodología de desarrollo de software que sigue un ciclo repetitivo de tres pasos:

1. **Red**: Escribir una prueba que falle para la funcionalidad que queremos implementar.
2. **Green**: Escribir el código mínimo necesario para que la prueba pase.
3. **Refactor**: Mejorar el código manteniendo todas las pruebas pasando.

Beneficios de TDD:

- Diseño más limpio y modular
- Cobertura de pruebas alta desde el inicio
- Documentación viva a través de pruebas
- Detección temprana de errores
- Facilita los cambios futuros

Sección 2: Ejemplo paso a paso de TDD

Ejemplo TDD: Método para verificar palíndromos en Java
Explicación del problema
Un **palíndromo** es una palabra, frase o secuencia que se lee igual hacia adelante que hacia atrás (ignorando espacios, signos de puntuación y mayúsculas/minúsculas). Ejemplos:

- "reconocer"
- "anita lava la tina"
- "A man, a plan, a canal: Panama"

Implementación con TDD paso a paso

Para implementar un verificador de palíndromos en Java utilizando el enfoque de **Desarrollo Guiado por Pruebas (TDD)**, seguiremos un ciclo de tres pasos: **Rojo**, **Verde** y **Refactorizar**. Este proceso nos asegura que escribimos código funcional y bien probado.
Aquí tienes una guía paso a paso utilizando **NetBeans** y **JUnit 4**.

1. Configuración del Proyecto en NetBeans

1. **Crea un Nuevo Proyecto:** En NetBeans, ve a File > New Project....
2. **Elige el Tipo de Proyecto:** Selecciona Java > Java Application.
3. **Configura el Proyecto:** Dale un nombre, por ejemplo, VerificadorPalindromo.
4. **Agrega la Dependencia de JUnit 4:**
    - Haz clic derecho en el proyecto y selecciona Properties.
    - Ve a la categoría Libraries.
    - Haz clic en Add Library... y selecciona JUnit 4.
    - Haz clic en Add Library y luego en OK.

2. Ciclo de TDD: Rojo, Verde, Refactorizar

Paso 1: Rojo (Escribir la Prueba que Falla)

En esta fase, escribimos una prueba que define lo que queremos que nuestro código haga. La prueba fallará inicialmente porque aún no hemos escrito el código de implementación.

1. **Crea una Clase de Prueba:**
    - Haz clic derecho en tu proyecto (Source Packages) y selecciona New > Other....
    - Elige JUnit > JUnit Test y asígnale el nombre PalindromoTest. NetBeans creará la clase con un esqueleto básico.
2. **Escribe la Primera Prueba:**
    - Añade un método de prueba para un caso sencillo, como "reconocer", que es un palíndromo.
    - Importa las aserciones de JUnit: import static org.junit.Assert.*;.

- Escribe el siguiente código dentro de la clase PalindromoTest:

Java
```
package verificadorpalindromo;

import org.junit.Test;
import static org.junit.Assert.*;

public class PalindromoTest {

    @Test
    public void testEsPalindromoConPalabraSimple() {
        // Se espera que "reconocer" sea un palíndromo, por lo tanto, true
        assertTrue(Palindromo.esPalindromo("reconocer"));
    }
}
```

- **Nota:** En este punto, el editor de NetBeans marcará Palindromo y esPalindromo con un error, ya que no existen. Esto es normal y parte del proceso.

3. **Ejecuta la Prueba:**
   - Haz clic derecho en PalindromoTest.java y selecciona Test File.
   - **Resultado:** La prueba fallará (en **rojo** 🔴) con un java.lang.Error (o similar) porque la clase Palindromo no existe. Esto confirma que nuestra prueba está configurada para fallar como se esperaba.

Paso 2: Verde (Escribir el Código Mínimo para que la Prueba Pase)

Ahora, escribiremos la menor cantidad de código posible para que la prueba que acaba de fallar, ahora pase. No nos preocupamos por la elegancia o la optimización en este momento.

1. **Crea la Clase de Implementación:**
   - Haz clic derecho en tu proyecto y selecciona New > Java Class.
   - Dale el nombre Palindromo.
2. **Escribe el Método Mínimo:**
   - Dentro de Palindromo.java, crea el método esPalindromo para que devuelva true solo para el caso que probamos.

Java
```
package verificadorpalindromo;

public class Palindromo {

    public static boolean esPalindromo(String texto) {
        // Implementación mínima para que la prueba pase
        return true;
    }
}
```

3. **Ejecuta las Pruebas de Nuevo:**
   - Ejecuta PalindromoTest.java otra vez.
   - **Resultado:** ¡La prueba debería pasar (en **verde** 🟢)! Esto significa que nuestro código cumple con la expectativa mínima definida por la prueba.

Paso 3: Refactorizar y Agregar Nuevas Pruebas

En esta fase, mejoramos el código sin cambiar su funcionalidad. Luego, repetimos el ciclo con una nueva prueba más compleja.

1. **Mejora el Código:**
   - La implementación actual siempre devuelve true, lo cual no es correcto para todos los casos. Implementemos la lógica real para verificar palíndromos.
   - La lógica debe:
     - Convertir la cadena a minúsculas (toLowerCase).
     - Eliminar espacios y signos de puntuación (replaceAll("[^a-z0-9]", "")).
     - Revertir la cadena y compararla con la original.

Java

```java
package verificadorpalindromo;

public class Palindromo {

    public static boolean esPalindromo(String texto) {
        // 1. Limpiar el texto (quitar espacios, puntuación y convertir a minúsculas)
        String textoLimpio = texto.toLowerCase().replaceAll("[^a-z0-9]", "");

        // 2. Invertir el texto
        String textoInvertido = new StringBuilder(textoLimpio).reverse().toString();

        // 3. Comparar el texto original limpio con el invertido
        return textoLimpio.equals(textoInvertido);
    }
}
```

2. **Agrega Nuevas Pruebas (Ciclo Rojo de Nuevo):**
    - Ahora que el código está refactorizado, agreguemos más casos a PalindromoTest para asegurar que la lógica funciona correctamente.

Java

```java
package verificadorpalindromo;

import org.junit.Test;
import static org.junit.Assert.*;

public class PalindromoTest {

    @Test
    public void testEsPalindromoConPalabraSimple() {
        assertTrue(Palindromo.esPalindromo("reconocer"));
    }

    @Test
    public void testNoEsPalindromo() {
        // Se espera que "hola" no sea un palíndromo
        assertFalse(Palindromo.esPalindromo("hola"));
    }

    @Test
    public void testEsPalindromoConFrase() {
        // Se espera que la frase sea un palíndromo ignorando espacios y puntuación
        assertTrue(Palindromo.esPalindromo("anita lava la tina"));
    }

    @Test
    public void testEsPalindromoConMayusculasYPuntuacion() {
        // Se espera que la frase con mayúsculas y signos de puntuación sea un palíndromo
        assertTrue(Palindromo.esPalindromo("A man, a plan, a canal: Panama"));
    }

    @Test
    public void testEsPalindromoConFraseVacia() {
        // La cadena vacía se considera un palíndromo
        assertTrue(Palindromo.esPalindromo(""));
    }
}
```

3. **Ejecuta Todas las Pruebas:**
    - Ejecuta PalindromoTest.java.

- **Resultado:** Todas las pruebas deberían pasar (en **verde** 🟢). Esto nos da confianza en que nuestra implementación es robusta y maneja los diferentes casos correctamente.

Con este enfoque, hemos construido una solución funcional y hemos verificado su corrección con un conjunto de pruebas que documentan el comportamiento esperado del método esPalindromo.

Sección 3: Ejercicios a desarrollar con TDD

1. Cálculo de área de un círculo: Área de un círculo = πr^2
   Caso de prueba 1: Radio positivo (radio = 5).
   Caso de prueba 2: Radio cero (radio = 0).
   Caso de prueba 3: Radio negativo (radio = -3).
   Caso de prueba 4: Radio grande (radio = 1000).
   Caso de prueba 5: Radio decimal (radio = 3.5).
2. Realizar una función para el Cálculo de número primo
   Caso de prueba 1: Número primo (n = 7).
   Caso de prueba 2: Número no primo (n = 6).
   Caso de prueba 3: Número negativo (n = -3).
   Número grande (n = 997).
   Caso de prueba 5: Número cero (n = 0).

   Realizar capturas para cada prueba y súbelo al clasroom en un documento pdf